МИНОБРНАУКИ РОССИИ

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ

ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Вычислительная математика»

Тема: «Метод бисекции»

Студент гр. 4342 Ковалев Е.С.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Изучить алгоритм метода бисекции (половинного деления) для нахождения корней нелинейных уравнений. Вычислить корень заданного уравнения с требуемой точностью. Исследовать скорость сходимости метода (зависимость числа итераций от точности) и проанализировать его устойчивость к погрешностям в вычислении функции.

Теоретический минимум

Метод бисекции опирается на теорему Больцано-Коши: если непрерывная функция $f(x)$ принимает на концах отрезка $[a,b]$ значения разных знаков (то есть выполняется условие $f(a)\cdot f(b) < 0$), то внутри этого интервала существует по крайней мере одна точка $c \in (a,b)$, в которой $f(c) = 0$.

Суть алгоритма заключается в итеративном сужении отрезка локализации корня. На каждом шаге текущий отрезок делится строго пополам точкой $x_{mid}=(a+b)/2$. Далее вычисляется значение функции в этой точке — f(xmid).

Возможны следующие варианты:

1. Если $f(x_{mid}) = 0$, то точный корень найден, и алгоритм завершается.

2. Если $f(a)\cdot f(x_{mid}) < 0$, это значит, что корень находится в левой половине. Границы обновляются: $b = x_{mid}$.

3. В противном случае корень находится в правой половине, и границы принимают вид: $a = x_{mid}$.

Процесс деления повторяется до тех пор, пока длина отрезка не станет удовлетворять заданной точности ε, то есть пока $|b-a| < 2\varepsilon$. Координата середины финального отрезка принимается за приближенное значение корня.

Метод гарантированно сходится для любых непрерывных функций, однако обладает относительно низкой скоростью сходимости. Количество необходимых итераций N для достижения точности ε оценивается логарифмической зависимостью: $N\approx\log_2(b-a)/\varepsilon$.

Выполнение работы

Из графика, построенного при помощи geogebra.org (рисунок 1), видно, что условие f(a)·f(b) < 0 для выполняется на отрезке [2.8, 3.2]

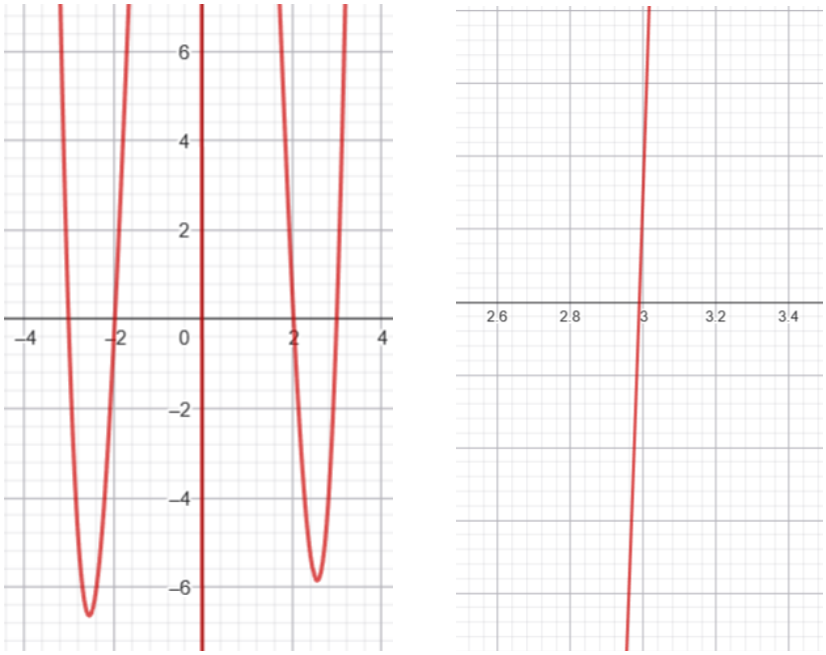

Рисунок 1 – График функции

В файле methods.cpp функция F(double x) была изменена в соответствии с вариантом 7: $f(x) = x^4 - 13x^2 + 36 + 1/x$.

Создан файл main.cpp (Приложение А), который вызывает функцию BISECT.

С помощью вычисленных результатов был построен график зависимости колчиества операций от Eps (Рисунок 2).

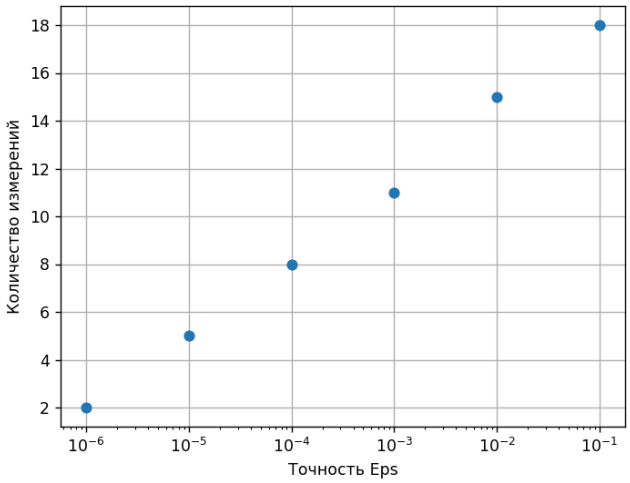

Рисунок 2 – График зависимости

Из графика видно, что уменьшение погрешности в 10 раз требует проведения в среднем 3 дополнительных итерации, что подтверждает логарифмическую оценку скорости сходимости метода.

Для исследование чувствительности к ошибкам исходных данных с помощью флагов было включено округление значений функции с шагом DELTA (имитация ошибок исходных данных).

Таблица 1 — Влияние ошибок округления на работу метода бисекции.

Значение DELTA	ε	x	Количество операций
10^{-1}	10^{-1}	2.9	2
10^{-1}	10^{-2}	2.9875	4
10^{-1}	10^{-3}	2.9875	4
10^{-1}	10^{-4}	2.9875	4
10^{-1}	10^{-5}	2.9875	4
10^{-1}	10^{-6}	2.9875	4
10^{-3}	10^{-1}	2.9	2
10^{-3}	10^{-2}	2.9875	5
10^{-3}	10^{-3}	2.989062	8
10^{-3}	10^{-4}	2.988672	9
10^{-3}	10^{-5}	2.988672	9
10^{-3}	10^{-6}	2.988672	9
10^{-5}	10^{-1}	2.9	2
10^{-5}	10^{-2}	2.9875	5
10^{-5}	10^{-3}	2.989062	8
10^{-5}	10^{-4}	2.988672	9
10^{-5}	10^{-5}	2.988672	9
10^{-5}	10^{-6}	2.988672	9
10^{-7}	10^{-1}	2.9	2
10^{-7}	10^{-2}	2.9875	5

10^{-7}	10^{-3}	2.989062	8
10^{-7}	10^{-4}	2.988672	11
10^{-7}	10^{-5}	2.988684	15
10^{-7}	10^{-6}	2.988673	18

Вывод

В ходе выполнения лабораторной работы был изучен и программно реализован метод бисекции. Корень уравнения $f(x) = x^4 - 13x^2 + 36 + 1/x$ найден с точностью до 10^{-6} ($x \approx 2.989299$). Анализ результатов показал логарифмическую зависимость числа итераций от заданной точности.

ПРИЛОЖЕНИЕ А

main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "methods.hpp"

#ifndef LEFT
#define LEFT 2.8
#endif
#ifndef RIGHT
#define RIGHT 3.2
#endif

int main() {
    int iterationsCount;
    std::cout << "Epsilon\t\t | Root (x)\t | Iterations\n";
    std::cout << "----------------------------------------------------\n";

    for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
        double root = BISECT(LEFT, RIGHT, epsilon, iterationsCount);
        std::cout << std::fixed << std::setprecision(6)
                  << epsilon << "\t | "
                  << root << "\t | "
                  << iterationsCount << '\n';
    }
    return EXIT_SUCCESS;
}
```

methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
```

```
#include <math.h>
#include <stdlib.h>

#ifndef ROUND
    #define ROUND 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND
extern double F(double x) {
    return std::pow(x, 4) - 13.0 * std::pow(x, 2) + 36.0 + 1.0 / x;
}
#else
extern double F(double x) {
    double val = std::pow(x, 4) - 13.0 * std::pow(x, 2) + 36.0 + 1.0 /
x;
    return Round(val, DELTA);
}
#endif

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;

    if (FLeft * FRight > 0.0)
    {
        puts("Неверное задание интервала\n");
```

```
        exit(1);
    }
    if (Eps <= 0.0)
    {
        puts("Неверное задание точности\n");
        exit(1);
    }
    N = 0;
    if (FLeft == 0.0)
        return Left;
    if (FRight == 0.0)
        return Right;

    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left);
        Y = F(X);
        if (Y == 0.0)
            return (X);

        if (Y * FLeft < 0.0)
            Right = X;
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    };
    return (X);
}
```

```
double Round(double X, double Delta)
{
    if (Delta <= 1E-9)
    {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return (Delta * (long((X / Delta) + 0.5)));
    else
        return (Delta * (long((X / Delta) - 0.5)));
}
```

methods.hpp

```
#ifndef METHODS_HPP
#define METHODS_HPP

extern double F(double);

#ifdef __NEWTON
extern double F1(double);
#endif

double Round(double X, double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
// double ITER(double X0, double Eps, int &N);
// double HORDA(double Left, double Right, double Eps, int &N);
// double NEWTON(double X, double Eps, int &N);

#endif
```